sv-tpu-core

Specification Document

Systolic Array Matrix Multiply Unit — 4×4, weight-stationary, int8

Owner: Jad | Phase 1 — Architecture Week (Jul 1–8, 2026) | Version 1.0

This is the single contract for the sv-tpu-core project. No RTL or UVM code is written until every team member (RTL guy, Atharva, Samarth, Jad) has read and approved this document. If something here is wrong, we fix it here first — never in the code.

Contents

0. How to Read This Document

This spec has three main parts, matching the three things that had to be locked before any code is written:

- **Part A — Design Spec:** the dimensions, timing, and behavior of the hardware.
- **Part B — Register (RAL) Spec:** exactly how the three control registers behave (this is what the UVM RAL model is built from).
- **Part C — Performance Spec:** how fast the design must run, and what the performance report must contain.
- **Part D — Formal Spec:** formal verification specifications for the 3 categories of assertions

Each part is written so that someone new to the project can understand it. Where a term is used for the first time, it is explained in plain language. A short glossary is at the end.

> **Quick guide — where each teammate's inputs are**
> Drawing the block diagram → use A.4 (module list), A.8 (signal map between modules), and A.9 (port lists).
> Defining port lists in the spec → A.9 already lays out every port; fill in or adjust there.
> Anything latency-related → Part C, especially C.2 (the 2N formula) and C.5 (who checks it).

> **The one rule that matters most**
> No one writes a single line of RTL or UVM until this document is committed and all four of us have approved it.
> If reality disagrees with this document during the build, stop and update this document first.

Part A — Design Spec - FULLY COMPLETED

(Note: Changed to DiP propagation, specify below)

This part defines what the chip is and how it behaves. It is the contract the RTL guy builds to, and the contract the verification team checks against.

A.1 What the Design Does (plain language)

The design multiplies two small grids of numbers together — a matrix multiply. This is the single operation that AI hardware spends most of its time on. Our design does it the way Google's TPU v1 does: with a systolic array, a grid of tiny multiply-add units that pass data to their neighbors every clock tick instead of going back and forth to memory.

Concretely: weights (one matrix) are loaded into the grid and held still. Activations (the other matrix) flow in from the left, one column per cycle. Each cell multiplies and adds into a running total. When the flow is finished, the totals are the answer and are read out.

A.2 Locked Design Decisions

These are fixed for v1.0. Changing any of them requires re-approval of this document by all four members.

Decision	Value	Why
Architecture	Weight-stationary systolic array	Weights load once and stay put; matches TPU v1; simplest correct dataflow.
Physical array size	4 × 4 processing elements (16 PEs)	Big enough to be a real systolic array, small enough for three new engineers to finish.
Parameterized on N	Yes — one constant changes 4×4 to 8×8	Shows the RTL is reusable; required for the resume claim 'scalable to N=8'.
Active size at runtime	DIM_REG selects N (1–4)	Lets one built design run 1×1 up to 4×4 without rebuilding.
Activation data type	int8 (signed, −128…127)	Matches TPU v1 quantized arithmetic.
Weight data type	int8 (signed, −128…127)	Matches TPU v1 quantized arithmetic.
Accumulator data type	int32 (signed)	Prevents overflow. Worst case 4×127×127 = 64,516, far below int32 max (~2.1 billion).
Control interface	AXI-Lite slave, 3 registers	Industry-standard control bus; every serious accelerator has one. Control only — not data.

A.3 Data Widths (exact bit counts)

Signal / value	Width	Signed?	Notes
Activation element	8 bits	Signed	One int8 value entering the array from the left.
Weight element	8 bits	Signed	One int8 value held stationary inside a PE.
PE accumulator	32 bits	Signed	Running multiply-add total inside each PE.
Output buffer element	32 bits	Signed	Final int32 result read by the testbench after done.
DIM_REG value (N)	3 bits	Unsigned	Holds 1–4 today; 3 bits leaves room up to 7 for an 8×8 build.

A.4 Module List and Boundaries

These are the RTL modules. The DUT boundary (what the testbench connects to) is mmu_top.sv.

Module	Role
mmu_top.sv	Top wrapper. Instantiates everything below. This is the DUT boundary.
axi_lite_slave.sv	The control panel — decodes the three AXI-Lite registers (see Part B).
mmu_controller.sv	The sequencer — the finite state machine that runs the computation (see A.5).
systolic_array.sv	The 4×4 grid of PEs. Parameterized on N. Fans pe_clear out to all PEs.
pe.sv	One processing element. One multiply-add. One pipeline register (see Part C).
output_buffer.sv	Holds the int32 results until the testbench reads them after done.
mmu_if.sv	The SystemVerilog interface bundle connecting the DUT to the testbench. Agreed by whole team.

A.5 Controller FSM and pe_clear Timing

The controller (mmu_controller.sv) is a finite state machine — it steps through a fixed sequence of states. Each state does one job, then hands off to the next.

State	What happens	Leaves when
IDLE	Waiting. Nothing runs.	CTRL_REG start bit is written (=1).
WEIGHT_LOAD	Weights are loaded into the PEs and held stationary.	All weights are loaded.
PE_CLEAR	pe_clear is asserted for exactly one cycle, zeroing every PE accumulator.	After exactly one cycle.
ACTIVATION_FLOW	Activations flow in from the left, one column per cycle. PEs multiply-add.	All activations have flowed through.
DONE	done bit (STATUS_REG) is set. Results are valid in the output buffer.	start is de-asserted / next run begins.

pe_clear — the signal most likely to cause a bug

pe_clear must be asserted for EXACTLY ONE cycle, only in the PE_CLEAR state.

It zeros every PE accumulator at the same time (it is fanned out to all 16 PEs).

If it is too short, too long, or mistimed, accumulator values leak from the previous run into the next one — a silent wrong-answer bug. Samarth's reset-stress and back-to-back sequences specifically hunt for this.

A.6 Timing Diagram (cycle by cycle, 2×2 example)

This shows a small 2×2 computation (N=2) so everyone can see the order of events. Each column is one clock cycle. Each row is one signal: a filled cell means the signal is active (high) that cycle. The pipeline cycle from Part C is included, which is why the final result lands at cycle 2N = 4.

Signal	0	1	2	3	4
State	IDLE	WLOAD	PCLR	AFLOW	DONE
start	1	0	0	0	0
pe_clear			1		
activation col				col 1	col 2
done					1

How to read it:

- **start:** Cycle 0 — software writes start = 1 (a one-cycle pulse via CTRL_REG). The FSM is in IDLE and now begins.
- **WEIGHT_LOAD:** Cycle 1 — weights are loaded into the PEs and held stationary.
- **PE_CLEAR:** Cycle 2 — pe_clear is high for exactly one cycle, zeroing every accumulator.
- **ACTIVATION_FLOW:** Cycles 3–4 — the activation columns flow in, one column per cycle (“col 1” then “col 2” for a 2×2). The PEs multiply-add.
- **DONE:** Cycle 4 — done goes high (STATUS_REG done bit = 1). The int32 results are now valid in the output buffer.

“col 1 / col 2” are the two activation columns of the 2×2 input matrix — column 1 enters the array first, column 2 the next cycle. For a 4×4 (N=4) the flow would be col 1 through col 4, and done would land at cycle 2N = 8.

A.7 Definition of Done (for the design)

The design is considered done when ALL of the following are true. This is the finish line everyone is building toward.

- A computation, once started, drives the FSM IDLE → WEIGHT_LOAD → PE_CLEAR → ACTIVATION_FLOW → DONE without getting stuck.
- pe_clear asserts for exactly one cycle in PE_CLEAR and zeros all accumulators.
- For any N from 1 to 4, the output buffer holds the correct int32 matrix-multiply result, matching the Python reference model.
- STATUS_REG done reads 1 only when results are actually valid; the testbench can poll it safely.

- Back-to-back computations produce correct results with no accumulator leakage between runs.
- A mid-computation reset returns the FSM to IDLE cleanly, and the next computation is still correct.

A.8 Signal Map Between Modules (for the block diagram)

This table is the source for the block diagram. Each row is one connection (an arrow) between two modules. Draw a box for every module in A.4, then draw these arrows between them.

From → To	Signal bundle	Width	Meaning
testbench → axi_lite_slave	AXI-Lite bus	standard	Read/write the three control registers (Part B).
axi_lite_slave → mmu_controller	start	1-bit	CTRL_REG start bit — tells the FSM to begin.
axi_lite_slave → mmu_controller	dim_n	3-bit	DIM_REG value — the active size N (1–4).
mmu_controller → axi_lite_slave	done	1-bit	Sets STATUS_REG done bit when results are valid.
mmu_controller → systolic_array	pe_clear	1-bit	One-cycle clear, fanned out to all 16 PEs.
mmu_controller → systolic_array	load_weight / flow_en	1-bit each	Control strobes for the WEIGHT_LOAD and ACTIVATION_FLOW phases.
testbench/data_agent → systolic_array	activations	N × 8-bit	int8 activation columns flowing in from the left.
weights source → systolic_array	weights	N × 32-bit	int8 weights loaded and held stationary.
systolic_array → output_buffer	results	N × 32-bit	int32 accumulated results draining out.
output_buffer → testbench/data_agent	read_data	32-bit	Results read out after done.

A.9 Port Lists (per module)

These are the agreed ports for each module. Names and widths here are the contract — RTL and testbench both use exactly these. (clk and rst_n are present on every synchronous module and are omitted from each list for brevity; assume active-low reset rst_n.)

pe.sv — single processing element

Port	Dir	Width	Meaning
activation_in	in	8	int8 activation from the left neighbor.
weight_in	in	8	int8 weight to hold stationary.
load_weight	in	1	Latch weight_in into the PE.

Port	Dir	Width	Meaning
pe_clear	in	1	Zero the accumulator (one cycle).
activation_out	out	8	Activation passed to the right neighbor.
accum_out	out	32	int32 running total (registered output — see Part C)
accum_in	in	32	Partial sum from the PE above; 0 for row 0

systolic_array.sv — N×N grid

Port	Dir	Width	Meaning
activations	in	N×8	One int8 per row entering from the left.
weights	in	N×N×8	int8 weights for the active N×N subset.
load_weight	in	1	Load strobe, fanned to all PEs.
pe_clear	in	1	Clear strobe, fanned to all PEs.
flow_en	in	1	Enable activation flow.
results	out	N×32	int32 results draining to the output buffer.

Note: internally `accum_in` of row 0 is tied to `0`, and that the module's `results` output is literally the bottom row's `accum_out` bus (N×32), not a separate reduction step.

mmu_controller.sv — FSM

Port	Dir	Width	Meaning
start	in	1	From CTRL_REG — begin a computation.
dim_n	in	3	From DIM_REG — the active size N.
load_weight	out	1	Drives WEIGHT_LOAD phase.
pe_clear	out	1	Asserted one cycle in PE_CLEAR.
flow_en	out	1	Drives ACTIVATION_FLOW phase.
done	out	1	To STATUS_REG — results valid.

axi_lite_slave.sv — control registers

Port	Dir	Width	Meaning
AXI-Lite channels	in/out	standard	Standard AXI-Lite slave signals (AW, W, B, AR, R).
start	out	1	Decoded CTRL_REG start bit.
dim_n	out	3	Decoded DIM_REG value.
done	in	1	From controller; drives STATUS_REG done bit.

Note to RTL guy on the port lists

These names are the agreed contract — if any name or width needs to change, change it here in A.9 first and tell the team, before touching code.

Output ports that feed the latency check (start, done) must be observable at mmu_top so both latency checkers in Part C can see them.

Part B — Register (RAL) Spec

This part defines the three control registers exactly. It is the contract for two things at once: how the RTL register file (axi_lite_slave.sv) behaves, and how the UVM RAL register model (mmu_reg_model.sv) is built. They must match this table exactly.

B.1 What 'RAL' means (plain language)

RAL stands for Register Abstraction Layer. Instead of a test driving raw bus wires to poke a register, the RAL model lets a test say something readable like reg_model.DIM_REG.write(4) and the framework handles the bus details. To build that model, the verification team needs the exact name, address, width, access type, and reset value of every register — which is what this part provides.

B.2 Register Map

Three registers, control only. The base address is assigned at integration; offsets below are relative to that base.

Register	Offset	Direction	Width	Reset	Purpose
DIM_REG	0x0	Write / Read	3-bit	0x0	Set matrix dimension N (1–4) for this computation.
CTRL_REG	0x4	Write / Read	1-bit	0x0	Bit 0 = start. Writing 1 triggers weight load then activation flow.
STATUS_REG	0x8	Read-only	1-bit	0x0	Bit 0 = done. Poll until 1 before reading the output buffer.

B.3 Per-Register Detail and Access Types

Access type is the single most important field for the RAL model. The terms used here:

- **RW (read-write):** software can write it and read it back. (DIM_REG, CTRL_REG)
- **RO (read-only):** software can only read it. Writes must have no effect — the hardware sets the value. (STATUS_REG)

Register	Field	Bits	Access	Reset	Behavior
DIM_REG	N	[2:0]	RW	0	Holds the active dimension 1–4. Values 0 and 5–7 are illegal for v1.0 (see B.4).
CTRL_REG	start	[0]	RW	0	Writing 1 starts a computation. Self-clearing behavior is defined in B.4.
STATUS_REG	done	[0]	RO	0	Set to 1 by hardware when results are valid. Writes are ignored.

B.4 Locked Register Rules

These rules are checked in verification. Both the RTL and the RAL model must obey them.

1. STATUS_REG is strictly read-only. A write to it must not change its value. (Formally proven by Atharva's AXI proof; also checked in the scoreboard.)
2. DIM_REG only accepts N in the range 1–4 for v1.0. Driving 0 or 5–7 is an illegal stimulus used by the error-injection sequences; the design must not produce a spurious result.
3. Writing start = 1 in CTRL_REG when the FSM is not IDLE is illegal (premature / double start). The error-injection sequences drive this; the scoreboard's negative check confirms no spurious output.
4. done in STATUS_REG reads 1 only when the output buffer genuinely holds valid results. The testbench relies on this to know when to read.
5. All three registers reset to 0.

RAL build note for Atharva and Samarth

mmu_reg_model.sv must declare DIM_REG (RW), CTRL_REG (RW), STATUS_REG (RO) with the reset values above.

The two built-in sequences uvm_reg_hw_reset_seq and uvm_reg_access_seq are run first during integration to shake out register-model and AXI-slave bugs before any directed test.

Part C — Performance Spec

This part defines how fast the design must run, in plain numbers, and exactly what the performance report (perf_report.md) must contain. It is the contract for Jad's performance verification work.

C.1 Why we verify performance (plain language)

Most student projects only check whether the chip gets the right answer. We also check whether it gets the right answer in the right number of clock cycles. Real hardware has to hit a timing target or it does not ship, so verifying performance — not just correctness — is what separates this project from a typical one.

PE Output — Registered (Design Decision)

The PE `activation_out` signal is registered through a flip-flop before being passed to the adjacent column. This is not a stylistic choice — it is a correctness requirement. The systolic array's timing model depends on activations taking exactly one clock cycle to traverse each column, which only holds if each PE introduces one pipeline stage. The controller's staggered activation feeding (row 0 on cycle 0, row 1 on cycle 1, etc.) is designed around this assumption, and the `result_latency` SVA property (`##[N*2:N*2]`) is only provable if propagation delay is exactly one cycle per column. A combinational pass-through would cause all columns to see the same activation in the same cycle, collapsing the pipeline, invalidating the stagger logic, and producing incorrect accumulation results — silently, with no timing violation to catch it. Registering the output also eliminates combinational glitch propagation across the array and gives the SVA assertion layer a well-defined, synchronous signal boundary to reason about. This is consistent with the original Google TPU v1 architecture and with the definition of a systolic array as a design where data flows through one register stage per clock cycle.

C.2 The Cycle-Latency Formula

Latency means: how many clock cycles pass between the activations starting to flow and the final result being ready.

An unpipelined weight-stationary N×N array finishes in 2N−1 cycles. Our PE has one pipeline register (see Part A and the note below), which adds exactly one cycle. So the locked latency contract for this project is:

latency = 2N cycles

This single number is shared by two checks (see C.5). Both must use 2N. The most likely integration mistake on the whole project is one checker still using 2N−1 after the pipeline is added — if a latency check fires unexpectedly, check this first.

Matrix size (N)	Unpipelined (2N−1)	Locked contract (2N)
N = 1 (1×1)	1 cycle	2 cycles
N = 2 (2×2)	3 cycles	4 cycles
N = 3 (3×3)	5 cycles	6 cycles

Matrix size (N)	Unpipelined (2N−1)	Locked contract (2N)
N = 4 (4×4)	7 cycles	8 cycles — worst case, most important to verify

C.3 What 'back-to-back throughput' means

Throughput means: when computations are run one immediately after another with no idle gap, how often does a new result come out? We measure it by running 10 or more computations back-to-back and recording the cycle count of each.

- Each individual computation must still meet its 2N latency (correctness of timing per run).
- Across the run, no result may take longer than its 2N budget, and accumulator values must not leak between runs (this overlaps with the back-to-back correctness check and is intentional).
- The measured throughput is reported against the theoretical best case so the number has meaning, not just a raw count.

C.4 What perf_report.md Must Contain

The performance report is a deliverable written after integration. It must contain, at minimum, all of the following. This is the checklist Jad writes to and the team reviews against.

6. A table of actual vs expected cycle counts for every matrix dimension N = 1, 2, 3, 4 (expected = 2N).
7. The back-to-back throughput result: how many computations were run, and the measured cycles-per-result vs the theoretical best.
8. A short explanation of the pipeline's latency/throughput tradeoff (one register stage adds one cycle of latency in exchange for a shorter per-stage critical path).
9. Any deviation between actual and expected, with its root cause and fix (cross-referenced to the matching BUGS.md entry).
10. A one-line statement of pass/fail: every dimension met its 2N target, yes or no.

C.5 How the latency property is verified — two checks, on purpose

Two team members verify latency, at two different levels. This is intentional defense-in-depth, not duplicated work. Both use the same 2N number.

Check	Owner	Level	What it does
latency_checker (in mmu_scoreboard.sv)	Samarth	Transaction-level	Records the cycle start fires and the cycle done asserts; asserts the gap equals 2N. Part of the per-transaction scoreboard check.
mmu_perf_checker.sv (bound to mmu_top)	Jad	Signal-level (SVA)	A bound SVA assertion that fires on the exact cycle a 2N violation occurs. Also measures back-to-back throughput, which the scoreboard does not.

Performance — Definition of Done

mmu_perf_checker.sv and the scoreboard latency_checker both pass on all legal stimulus for N = 1 to 4, and both use 2N.
mmu_perf_checker.sv is part of make regress (not a separate manual step).
perf_report.md contains all five items in C.4.
Each checker has been shown to actually fire on a deliberately injected one-cycle timing error (an unexercised checker is not a passing checker).

C.6 Latency Specification — The 2N Contract (Shared Reference)

Owner: Samarth (spec author). Shared by: latency_checker in mmu_scoreboard.sv and mmu_perf_checker.sv. This subsection is the single source of truth for the latency number. Every assertion, every checker, and every README claim about latency derives from here. If this number changes, both checkers change simultaneously in the same PR — never one without the other.

The formula

Measured from: the first cycle of ACTIVATION_FLOW state (the first cycle when act_in is asserted at the array input).

Measured to: the cycle when STATUS_REG bit 0 (done) asserts high.

This is a hard contract, not a target. Off by one cycle in either direction is a bug.

Where 2N comes from

Two contributions of N cycles each:

- **Activation propagation: Row 0's first activation enters PE[0][0] on cycle 1. That value reaches PE[0][N-1] on cycle N. The last PE in each row receives its first input N cycles after activation flow starts.**
- **Accumulation: Once a PE receives its first activation, it needs N cycles to accumulate all N dot product terms — one new activation per cycle.**

The pipeline register (see C.1) adds exactly one cycle and is absorbed into this formula uniformly across all dimensions. Do not write 2N-1+1 anywhere. The locked number is 2N.

Verification table — values both checkers must use

Matrix size (N)	Unpipelined (2N−1)	Locked contract (2N)
N = 1 (1×1)	1 cycle	2 cycles
N = 2 (2×2)	3 cycles	4 cycles
N = 3 (3×3)	5 cycles	6 cycles
N = 4 (4×4)	7 cycles	8 cycles — worst case, most important to verify

Bugs this contract is designed to catch

- pe_clear holds for more than one cycle: DONE arrives late.
- Controller FSM off-by-one in ACTIVATION_FLOW cycle count: DONE arrives early or late.
- Pipeline register not accounted for in FSM DONE timing: DONE arrives one cycle early — this is the 2N-1 bug.
- Accumulator leakage between back-to-back runs: wrong result; latency may appear correct but scoreboard fires.

Change control

If the latency contract changes from 2N to anything else, update ALL FIVE in the same PR:

- This subsection (C.6)
- mmu_scoreboard.sv (latency_checker)
- mmu_perf_checker.sv
- perf_report.md expected column
- README latency claim

Do not change any one of them alone.

Section D:

Formal Verification Plan — JasperGold - FULLY COMPLETE

Owner: Atharva (verification lead) — sole team member with JasperGold training.
Tool: Cadence JasperGold. Formal property files are run exclusively through

JasperGold and never through Xcelium. The two toolchains are completely separate flows.

Golden rule: Every property below proves something that simulation cannot prove regardless of how many tests are run. The interview talking point is: *"We used formal verification for properties where exhaustive simulation is mathematically impossible. Each proof holds for all possible input combinations, not just the ones we thought to generate."*

File Structure

```
formal/
├── pe_formal.sv          ← Proof 1: overflow impossibility property
├── axi_formal.sv         ← Proof 2: AXI protocol compliance properties (x3)
├── mmu_formal.sv         ← Proof 3: 2×2 functional correctness property
├── pe_overflow.tcl       ← JasperGold script for Proof 1
├── axi_compliance.tcl    ← JasperGold script for Proof 2
├── twobytwo_correct.tcl  ← JasperGold script for Proof 3
└── results/
    ├── proof1_overflow_PROVEN.png    ← Screenshot committed before v1.0 tag
    ├── proof2_axi_PROVEN.png
        └── proof3_2x2_PROVEN.png
```

Proof 1 — Accumulator Overflow Impossibility - COMPLETE

File: `pe_formal.sv` | **Bound to:** `pe.sv` | **TCL:** `pe_overflow.tcl`
Timeline: Jul 28 – Aug 1 | **Expected result:** PROVEN

What it proves (prose): Given that both `activation_in` and `weight` are constrained to the int8 range (−128 to 127), and given that the accumulator performs at most N=4 multiply-accumulate operations before being cleared by `pe_clear`, the int32 accumulator can never overflow. Specifically: the maximum possible accumulated value is 4 × (127 × 127) = 64,516, and the minimum is 4 × (−128 × 127) = −65,024. Both are well within the int32 range of approximately −2.1 billion to +2.1 billion. This property holds for every possible int8 input combination — not just those generated by simulation.

Formal assumptions to constrain the proof:

- `activation_in` is within [−128, 127]
- `weight` is within [−128, 127]
- The accumulator is cleared (`pe_clear` asserted) before each new computation
- At most 4 accumulation cycles occur per computation (N=4 bound)

Property statement in prose: *After any sequence of at most 4 accumulation cycles with int8 inputs and int8 weights, the value of* `acc` *is representable within a 32-bit signed integer and has not wrapped around.*

If a counterexample appears: The input constraints in the TCL are wrong — iterate on the assumptions, not on the SV property. The math guarantees this proof will be PROVEN under correct constraints.

Proof 2 — AXI-Lite Protocol Compliance - COMPLETE

File: `axi_formal.sv` | **Bound to:** `axi_lite_slave.sv` | **TCL:** `axi_compliance.tcl`
Timeline: Aug 1 – Aug 3 | **Expected result:** PROVEN × 3

What it proves (prose): The AXI-Lite slave correctly implements the protocol under any possible master behavior — not just the well-behaved stimulus generated by the UVM agent. The master is treated as unconstrained; the proof holds for any sequence of valid and invalid master transactions.

Three independent properties are proven:

Property 2a — AWVALID stability: Once the master asserts AWVALID (write address valid), it must hold AWVALID asserted continuously until the slave responds with AWREADY. The slave must never observe AWVALID drop before AWREADY is given. *In prose: if AWVALID is high and AWREADY is not yet high, then AWVALID must remain high on the next cycle.*

Property 2b — STATUS_REG read-only enforcement: A write transaction targeting the STATUS_REG address (offset 0x08) must never alter the value stored in STATUS_REG. The register is read-only by architectural definition; no master action can corrupt it. *In prose: for any write transaction where the write address equals 0x08, the value of STATUS_REG before and after the transaction is identical.*

Property 2c — No spurious write response: The slave must never assert BVALID (write response valid) unless a complete write transaction — both the address phase (AWVALID/AWREADY handshake) and the data phase (WVALID/WREADY handshake) — has already completed. No phantom acknowledgements are permitted. *In prose: BVALID can only be asserted on a cycle that follows a completed AWVALID/AWREADY and WVALID/WREADY handshake pair.*

If a counterexample appears on 2a or 2c: This is a real RTL protocol bug in `axi_lite_slave.sv`. Log in BUGS.md and fix the RTL immediately — do not relax the property.
If a counterexample appears on 2b: This is a read-only enforcement bug. The slave is permitting writes to STATUS_REG to take effect. Fix the decode logic in `axi_lite_slave.sv`.

Proof 3 — 2×2 Functional Correctness *(Crown Jewel) - COMPLETE*

File: `mmu_formal.sv` | **Bound to:** `mmu_top.sv` (with DIM_REG = 2) | **TCL:** `twobytwo_correct.tcl`
Timeline: Aug 3 – Aug 10 (first attempt Aug 3–5; fallback continues into Phase 4 if needed)
Expected result: PROVEN (or k-induction result — see fallback ladder)

What it proves (prose): With DIM_REG set to 2, for every possible combination of int8 values in the four activation inputs and the four weight inputs — all 2^32 combinations — the four int32 outputs produced by the design after 2N = 4 cycles exactly equal the result of 2×2 integer matrix multiplication as defined by the standard dot-product formula. This is not "the output matched the Python reference model on every test we ran." It is a mathematical proof that the hardware is correct for all inputs simultaneously. This is the level of correctness guarantee that tape-out sign-off requires.

Formal assumptions to constrain the proof:

- DIM_REG is held at 2 for the duration of the proof
- All four activation inputs are free int8 variables (unconstrained within [−128, 127])
- All four weight inputs are free int8 variables (unconstrained within [−128, 127])
- The computation begins from a known-clean state (accumulators zeroed by `pe_clear`)
- The property is checked at the cycle when `done` asserts (2N = 4 cycles after activation flow begins)

Property statement in prose: *When DIM_REG = 2 and a computation completes (done asserts), output[row][col] equals the sum over k of (activation[row][k] × weight[k][col]), computed in signed integer arithmetic, for all four (row, col) combinations.*

Fallback ladder — in priority order:

1. **k-induction:** If the full state space causes the engine to return UNKNOWN, apply k-induction. This proves the property holds for all states reachable within k steps. The result is still a formal proof. Resume line: *"bounded formal correctness proof via k-induction."*
2. **1×1 fallback:** If k-induction is still inconclusive, reduce scope to a single PE: formally prove that one PE produces `acc = activation × weight` correctly for all int8 input combinations. The state space is minimal and this will be PROVEN. Still a meaningful result.
3. **Honest documentation:** If the 2×2 proof remains UNKNOWN after full effort in Phase 4, document the attempt, the state space size, and the fallback results honestly in the README. Attempting a hard proof and documenting the boundary is a sign of formal maturity, not failure.

What Distinguishes Formal From the SVA Simulation Assertions

The SVA assertions in `/sva/` (bound via Xcelium simulation) check properties on the specific inputs that the UVM sequences generate. If a bug exists for an input combination that no sequence ever produces, simulation will never find it.

The JasperGold formal proofs make no assumptions about which inputs arrive. The tool exhaustively explores all reachable states and either proves the property holds universally or produces a specific counterexample input that violates it. This is why formal is the right tool for overflow guarantees, protocol invariants, and functional correctness — and why "PROVEN" on a formal result means something categorically different from "all tests passed."

Approvals & Glossary

Sign-off — all four must approve before any code is written

Team member	Role	Approved (Y/N)	Date
RTL guy	RTL design	Y	6/06
Atharva	Verification lead — formal proofs, UVM env	Y	6/06
Samarth	Verification — scoreboard, coverage, adversarial sequences	Y	6/06

Team member	Role	Approved (Y/N)	Date
Jad	Performance verification + pipeline (spec owner)	Y	6/06

Glossary

Term	Plain meaning
Systolic array	A grid of tiny multiply-add units that pass data to neighbors each cycle instead of using memory.
PE (processing element)	One cell in the grid. Does one multiply-add per cycle.
Weight-stationary	Weights are loaded once and held still; activations flow past them.
MAC	Multiply-Accumulate: total = total + (a × b). The one operation a PE does.
int8 / int32	8-bit / 32-bit signed integers. Inputs are int8; the running total is int32 to avoid overflow.
AXI-Lite	A simple industry-standard control bus used to read/write the three registers.
RAL	Register Abstraction Layer — lets tests use register names instead of raw bus wires.
FSM	Finite State Machine — the controller that steps through IDLE → … → DONE.
pe_clear	A signal that zeros every PE accumulator for exactly one cycle before a new run.
Latency	Cycles from activations starting to flow until the result is ready. Locked at 2N here.
Throughput	How often a new result comes out when runs are packed back-to-back.
SVA	SystemVerilog Assertions — rules checked automatically every cycle during simulation.
DUT	Design Under Test — the thing being verified. Here, mmu_top.sv.